Chapter 2: Debugging and Error Handling

Almost all code of any significant size and complexity initially contains errors. You can use the Visual Studio debugger to eliminate logical errors from your applications. The debugger is a powerful tool that provides several easy-to-use techniques for identifying and removing errors from your code.

In addition to errors, exceptional conditions caused by an external software or hardware environment might also affect the performance of an application. You need to anticipate and handle possible failures by adding necessary exception-handling code to ensure smooth program execution.

In this chapter, you will look at some important features of the Visual Studio debugger. You will also learn to write code to handle errors and exceptions in your applications.

Objectives

After completing this chapter, you will be able to:

- Use the Visual Studio debugger to identify and eliminate errors.
- Use the Edit and Continue feature to simplify debugging.
- Write functions with built-in error-handling code.
- Use the C++ exception-handling technique to handle run-time exceptions.
- Describe and use the various types of MFC exception classes.

30 Debugging

An error is the failure of a function or process to carry out its assigned task because of some directly controllable parameter or condition. For example, **CView::GetDocument** returns a NULL pointer if the associated view is not attached to a document. You can use the Visual Studio debugger to correct the errors in your code.

This section begins with an overview of the Visual Studio debugger features. You will read about setting and managing breakpoints, viewing and modifying variables, stepping through your application, and interpreting the call stack. You will then learn how to use the Edit and Continue feature, which enables you to modify your code and continue execution of the revised code without terminating your debugging session.

This section includes the following topics:

- Overview of the Visual Studio Debugger
- Using the Edit and Continue Feature

Notes

31 Overview of the Visual Studio Debugger

The Visual Studio debugger provides a variety of features and tools to help you track down errors in your code.

Visual Studio Debugger Features

The debugging tools in Developer Studio enable you to test your C++, MFC, and mixed-language applications.

You can use the debugger to:

- Set breakpoints.
- Step through code.
- Monitor variables, registers, and memory.
- View assembly code and the call stack.
- Modify code and variable values.

To see a demonstration of how to use the debugger, click this icon.
 (CD-ROM plays the demonstration, "Using the Visual Studio Debugger.")

Visual Studio Debugger Interface

The Visual Studio debugger interface is easy to use and consists of the following elements:

- Debugger menus

  You can find the commands related to debugging on the **Build** menu, the **Debug** menu, the **View** menu, and the **Edit** menu.
- Debugger toolbar

  Developer Studio provides the **Debug** toolbar to make common debugging tasks easier.
- Debugger windows

  During a debugging session, the debugger displays debugging information in different windows according to the type of information. You can view these windows by clicking **Debugging Windows** on the **View** menu.
- Debugger dialog boxes

  You can use the debugger dialog boxes to manipulate breakpoints, variables, threads, and exceptions.

Notes

32 Using the Edit and Continue Feature

Visual C++ includes an Edit and Continue feature in the debugger. This feature enables you to change the source code during a debugging session and continue debugging without terminating the debugging session. After you make a change using this feature, you do not need to terminate the debugging process and rebuild the application. Your changes are recompiled and applied to the executing debugger in memory.

To see a demonstration of how to use Edit and Continue, click this icon.

 (CD-ROM plays the demonstration, "Using Edit and Continue.")

Enabling Edit and Continue

You enable or disable the Edit and Continue feature by changing the settings in the **Options** dialog box. Edit and Continue is enabled by default.

▶ **To enable Edit and Continue**

1. On the **Tools** menu, click **Options**.
2. On the **Debug** tab in the **Options** dialog box, select **Debug commands invoke Edit and Continue**.
3. On the **Project** menu, click **Settings**.
4. On the **C/C++** tab in the **Project Settings** dialog box, select **Program Database for Edit and Continue** in the **Debug Info** list.

You need to fully rebuild the project before you start the first debugging session in which you want to use Edit and Continue.

Using Edit and Continue

After modifying source code, invoke the **Step** or **Go** debugger commands to initiate Edit and Continue. If the code edit is not supported, you will be prompted to rebuild the application. If the edit is supported, the changes will be recompiled and reflected in the executing application.

Edit and Continue does not support edits to the following:

- Header files
- C++ class definitions
- Function prototypes
- Global/static code

For more information, search for "Edit and Continue" in MSDN Library Visual Studio 6.0.

Notes

33 Handling Errors and Exceptions

This section covers some effective techniques for handling errors and exceptions. It begins by describing the error-handling code that you can build into your functions to ensure smooth program execution. When a critical error occurs, the error-handling code stops execution of the application.

This section also describes the exception-handling technique that you can use in your MFC-based applications. The section ends with a discussion on the MFC exception classes.

This section includes the following topics:

- Writing Robust Functions
- Handling Exceptions
- Exception-Handling Process
- MFC Exception Classes

Notes

34 Writing Robust Functions

Almost any function, including Win32 and MFC routines, can fail. This is especially true in multitasking environments, such as Windows or Windows NT®, where applications share limited system resources. You should anticipate this possibility, and write your programs accordingly.

To properly code an application, you must anticipate possible errors by checking return values and taking appropriate action when a function fails.

Reporting Errors

Many MFC functions return special values to indicate that an error has occurred. In your functions, you should implement this mechanism to report a failure back to the calling function.

Two common techniques exist for reporting errors:

- Returning values

  Set the return value to FALSE, NULL, 0xFFFFFFFF, or –1 to indicate a function failure.

  The following example code shows the definition of a function that returns a value indicating success or failure:

```
int CMyView::GetLastWord()
{
    ...
    if ((m_nWordCount < m_nCurrentWordNumber) || (m_nWordCount <= 0))
        return -1;
    ...
}
```

- Setting global error codes

  Set a global error code indicating the type of error that occurred. Call the **SetLastError** function to set a global error code when a function fails.

When implementing error-handling code in your application, you must consider the balance between extensive error-checking and program execution speed when implementing error-handling code in your application.

Notes

Handling Errors

Handling errors involves writing code that will alter program execution when a called function reports an error. How you check for errors depends on how the function indicates an error:

- If the function returns a value, check the return value of the function.

  The following example code shows how you can decide the course of action at run time by checking for the return value:

```
...
int b = pView.GetLastWord();
if (b == -1)
    m_ID = ProcessEmptyDocument();
else
    m_ID = ProcessDocument();
...
return;
```

- If the function sets a global error code, call the **GetLastError** function to determine whether an error occurred.

Notes

35 Handling Exceptions

Unlike an error, an exception is an abnormal—and often unexpected—event that changes the normal flow of processing and may require special handling by the operating system. Exceptions are raised as the result of hardware or software events, which are usually undetectable during development and are beyond the direct control of the program.

Accessing a bad memory address and dividing by zero are examples of hardware exceptions. Passing an invalid parameter or insufficient memory are examples of software exceptions. You can respond to these exceptions by using special C++ syntax called exception handling.

In exception handling, the operating system and the process in which the exception occurred cooperate to resolve the exception. Exceptions are generated and handled on a thread-by-thread basis. When an exception is raised, the operating system finds an appropriate handler, and passes control to the handler to resolve the exception or terminate the process or program.

When no matching handler is found, a predefined run-time **terminate** function is called. The default action of **terminate** is to call **abort**. You can use **set_terminate** to provide your own termination routine instead.

C++ Exception-Handling Technique

C++ exception handling is the preferred method when programming C++ and MFC-based applications. It is part of the ANSI/ISO language standard, so it is available in all modern C++ implementations on all supported platforms.

C++ exception-handling code consists of three major parts: the **try** (guard) block, the **throw** statement, and the **catch** block.

In the following example, the **try** block attempts to write the headers. The catch block then handles a specific file exception, and passes all other exceptions on to the outer block with the throw macro:

```
try
{
      // Write the file header
      file.Write((LPSTR)&bmfHdr, sizeof(BITMAPFILEHEADER));
      //
      // Write the DIB header and the bits
      file.WriteHuge(lpBI, dwDIBSize);
}
catch (CFileException* e)
   {
      ::GlobalUnlock((HGLOBAL) hDib);
      throw;
   }

::GlobalUnlock((HGLOBAL) hDib);
   return TRUE;
```

Notes

Using the try Block

The **try** block is any block of code prefaced with the **try** statement and surrounded by curly brackets (**{ }**). To create the most robust application, any code or function calls that may generate exceptions should be contained within a **try** block.

Using the throw Statement

The **throw** statement transfers control to the exception-handler that matches the type of exception specified in the **throw** argument.

Using a catch Handler

The **catch** block is composed of a **catch** expression, immediately followed by a handler. The **catch** expression defines which exception (or exceptions) the immediately following exception handler should handle; it filters the passing exceptions. Once an exception is caught, control passes to the exception handler.

The **catch** handler has two major purposes:

- Handle the specific exception or exceptions specified by the **catch** statement appropriately.
- Delete the exception object it if was dynamically created.

A handler for a base class also handles classes derived from that class. The **catch** clause catches exceptions of any type.

To associate different code blocks with different exceptions, a single **try** block can be followed by one or more **catch** blocks. It is also possible to nest **try** blocks.

Function Exception Specifications

For documentation and software-quality purposes, if a function is capable of throwing an exception, you should specify so in its declaration. For example, a nondefault **CPen** constructor has been declared as follows:

```
CPen(int nPenStyle, int nWidth, COLORREF crColor)
    throw (CResourceException);
```

This declaration indicates that this function may throw an exception of type **CResourceException**.

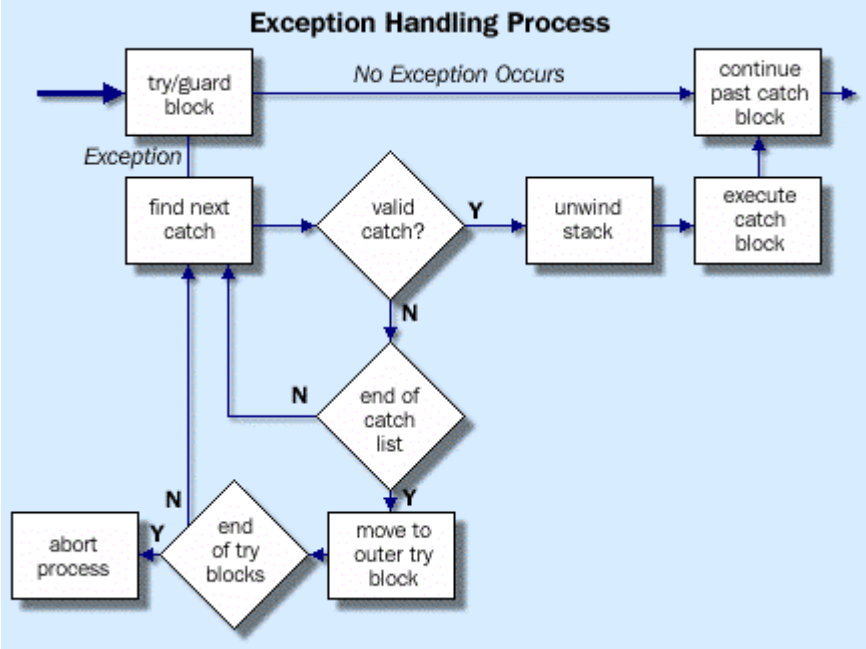
For more information about handling exceptions, search for "Exception Handling" in MSDN Library Visual Studio 6.0.

Notes

36 Exception-Handling Process

Exception handling follows a specific path. To see an illustration of the exception-handling process, click this icon.

(CD-ROM displays the following illustration.)



The following events occur in the exception-handling process:

- **Entering the try block**

  Control reaches the guarded section (within the **try** block) by normal flow control. The guarded section begins executing. A **try** (guard) block can throw more than one exception, though only one exception can be thrown at a time. A **catch** block can throw its own exceptions, or rethrow the caught exception.

- **If no exception is thrown**

  If no exception is thrown during execution of the guarded section, the **catch** blocks that follow the **try** block are not executed. Execution continues at the statement after the last **catch** block following the **try** block in which the exception was thrown.

- **If an exception is thrown**

  If an exception is thrown during execution of the guarded section (or in any routine the guarded section calls directly or indirectly), an exception object is created from the object created by the **throw** operand. (This implies that a copy constructor may be involved.)

- **Searching for a catch handler**

  If an exception is thrown, the compiler looks for a **catch** clause that can handle an exception of the type thrown. The **catch** handlers are examined in order of their appearance following the **try** block. If no appropriate handler is found, the next enclosing **try** block is examined. This process continues until the outermost enclosing **try** block is examined.

Notes

- **If a matching catch handler is not found**

  If a matching handler is still not found, or if another exception occurs before or during stack unwinding (but before the handler gets control), the **terminate** function is called.

- **If a matching catch handler is found**

  If a matching **catch** handler is found, the parameter is initialized to refer to the exception object.

- **Unwinding the stack**

  After the parameter is initialized, the process of unwinding the stack begins. This involves the destruction of all local objects that were constructed (but not yet destroyed) between the beginning of the **try** block associated with the **catch** handler and the exception's throw site. Destruction occurs in reverse order of construction. The **catch** handler is executed and the program resumes execution following the last handler (that is, the first statement or construct that is not a **catch** handler).

Note Control can enter a **catch** handler through a thrown exception only—never by means of a **goto** statement or a **case** label in a **switch** statement.

Notes

37 MFC Exception Classes

When you use C++ exceptions in an MFC-based application, you write exception handlers that use pointers to **CException** or **CException**-derived objects that can be thrown by the MFC framework or by your application.

Handler functions are polymorphic; a handler written for a base-class type will catch handlers of a derived type. To extend the exception architecture, you also can derive new exception classes from the ones provided.

Extensions to the CException Class

MFC provides the following predefined extensions to the **CException** class.

Exception class	Description	Thrown by
CMemoryException	Out of memory	**new** operator
CFileException	File exceptions	**CFile** functions
CArchiveException	Archive/serialization exception	**CArchive** functions
CNotSupportedException	Response to request for unsupported service	Any function that requests an unsupported service
CResourceException	Windows resource-allocation exception	Any function that requests a Windows resource
CDBException	ODBC database exceptions	Member functions of **ODBC** database classes
CDaoException	DAO database exceptions	Member functions of **DAO** database classes
COleException	OLE exceptions	Functions performing OLE operations
COleDispatchException	OLE dispatch (automation) exceptions	Functions performing OLE automation-related operations
CUserException	Exception that alerts the user with a message box, then throws a generic **CException**	Functions that perform operations that are likely to fail
CInternetException	Internet exception	Any function that fails in an Internet-related operation

The base class, **CException**, provides two member functions, **ReportError** and **GetErrorMessage**, to assist in handling exceptions. Many of the derived classes provide additional functionality.

For more information about **CException**, search for "CException" in MSDN Library Visual Studio 6.0.

Notes

Throwing MFC Exceptions

You can use existing MFC exception classes to throw exceptions. MFC provides a number of helper functions, such as **AfxThrowMemoryException**, to throw a preallocated exception object of the appropriate type.

The following code shows the use of these helper functions:

```
void CSomeClass::SomeFunction()
{
    char* p1 = (char*) malloc(SOME_SIZE);
    if (NULL == p1)
        AfxThrowMemoryException();
    ...
    free ( p1 );
}
```

Last Chance Exception Handling

If you do not handle an exception, program execution will abort in an undesirable manner. To prevent this behavior, you can modify the C++ **terminate** function to call a function other than **abort**. For more information about how to implement this technique, search for "set_terminate" in MSDN Library Visual Studio 6.0. The MFC Library provides the corresponding functions **AfxTerminate** and **AfxSetTerminate**, which you can use instead in an MFC-based application.

Notes

41 Self-Check Questions

1. Which type of edit is supported by Edit and Continue?

- A. Editing header files
- B. Editing function prototypes
- C. Editing functions on the call stack
- D. Editing C++ class definitions

2. Which keyword transfers control to the matching exception handler?

- A. **abort**
- B. **try**
- C. **throw**
- D. **catch**

3. What is the base class for exception types in MFC?

- A. **CMemoryException**
- B. **CResourceException**
- C. **CUserException**
- D. **CException**

4. What is the purpose of the stack unwinding process?

- A. Delivery of the exception object to the matching **catch** handler.
- B. Destruction of automatic objects in the order in which they were created.
- C. A call to the predefined **terminate** function.
- D. Destruction of local objects in the affected part of the stack in the reverse order.

Notes